

Optimizing Hot Page Scheduling in CXL Memory via Multilayer Perceptron-Based Prediction

1st Yuheng Xia

College of Computer Science
and Technology
Xidian University
China, Xian

qdkj200206054433@163.com

2rd Shaofeng Li*

College of Computer Science
and Technology
Xidian University
China, Xian

*Correspondence : lishaofeng@xidian.edu.cn

3rd Jinhui Liu

College of Computer Science
and Technology
Xidian University
China, Xian

jhliu@mail.xidian.edu.cn

4th Yanpeng Wang

China State Shipbuilding Corporation
716 Research Institute
China, Lianyungang
wyp716@126.com

Abstract—This paper presents a system-level framework for intelligent page migration in CXL-extended heterogeneous memory architectures. The design features a closed-loop runtime mechanism that integrates online feature extraction with adaptive parameter inference using a multilayer perceptron (MLP) model. To emulate architectural asymmetry between DRAM and CXL memory domains, we construct a virtualized system-level NUMA architecture using QEMU, enabling cross-node data movement, parameter control, and memory profiling. Experimental evaluation under representative workloads and DRAM-to-CXL ratios demonstrates that the proposed framework achieves up to 22.48% improvement in bandwidth and 19.29% reduction in latency compared to static migration baselines. These results validate the architectural effectiveness and adaptability of learning-driven memory orchestration in disaggregated system environments.

Index Terms—Compute Express Link (CXL), NUMA architecture, QEMU emulator, Hot page migration algorithm, Adaptive dynamic optimization.

I. INTRODUCTION

With the growing architectural complexity of heterogeneous computing systems and the proliferation of disaggregated memory solutions, memory access bottlenecks have emerged as a critical system-level chal-

lenge. Traditional host-to-host communication largely depends on the TCP/IP protocol stack [1] [2]. To address these issues, Compute Express Link (CXL) has emerged as a next-generation interconnect standard. Built on PCIe's physical layer, CXL offers cache coherence and memory semantics for low-latency, high-bandwidth communication among processors, accelerators, and memory devices [3]. As modern parallel workloads increasingly rely on dynamic and fine-grained memory orchestration, CXL-based systems provide architectural foundations for decoupled memory hierarchies and adaptive page scheduling across heterogeneous memory domains.

To address the lack of deployable CXL hardware and tooling, we develop a dual-node NUMA simulation environment using the QEMU virtualization platform, where local DRAM and emulated CXL memory constitute distinct memory domains. Building upon this setup, we propose a hot page migration strategy enhanced by a multilayer perceptron (MLP) model, which predicts critical migration parameters—including the maximum number of pages to migrate, the access difference threshold, and the migration interval—based on runtime system characteristics. The system architecture is designed with a modular workflow encompassing feature extraction, parameter prediction, strategy execution, and result logging. The proposed solution achieves latency reduction and bandwidth enhancement

This paper supported by “the Fundamental Research Funds for the Central Universities XJSJ23031”, the Natural Science Basic Research Program of Shaanxi(Grant No. 2024JC-YBQN-0732).

under varied workloads, demonstrating its viability for intelligent memory management in future distributed computing infrastructures.

II. ARCHITECTURAL BACKGROUND

A. CXL-Based Memory Architectures

CXL targets coherent, shared-memory access across heterogeneous devices, unifying CPU caches, host memory, and device-local memory in servers [4]. CXL 1.0 defines three protocols—CXL.io, CXL.cache, and CXL.mem—while CXL 2.0 adds pooling and switching for multi-host/device resource sharing with backward compatibility [5]. A Linux-based study demonstrates a CXL-enabled hybrid DRAM–SSD memory pool under cloud workloads [6]. Figure 1 illustrates a CXL 2.0 memory-pooling architecture where multiple hosts (H1–H#) access shared devices (D1–D#) via a unified switch managed by a fabric controller [7].

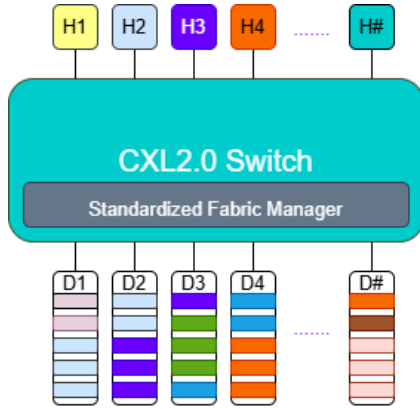


Fig. 1. CXL 2.0 Memory Pooling Structure Diagram

With the release of PCIe 6.0, CXL 3.0 further enhances scalability and reduces system-level latency by introducing support for direct peer-to-peer communication and multi-level topologies [8] [9]. Figure 2 illustrates a multi-stage switching topology under the CXL 3.0 architecture.

Figure 3 summarizes the three CXL device types. Type 1 devices typically have no local memory and rely on host memory for computation. Type 2 devices, such as GPUs and AI accelerators require access to both host and device memory [10]. Type 3 devices serve exclusively as memory expanders, typically forming DRAM- or NVM-based memory pools [11].

Recent CXL research advances memory pooling and page migration across clouds and composable fabrics: Type-3-based DRAM pools with ML-guided

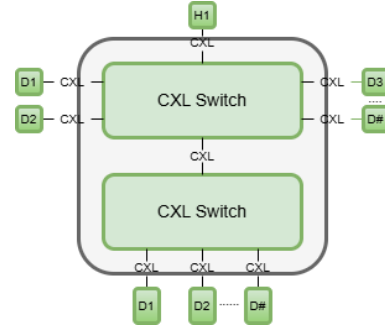


Fig. 2. CXL 3.0 Topology Structure

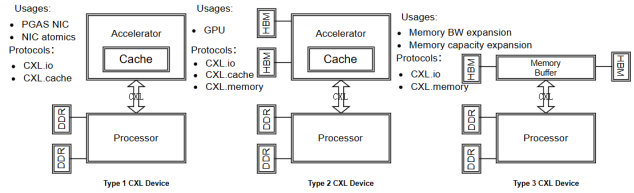


Fig. 3. Three Types of CXL Devices

local/pooled allocation [12]; a CXL-enabled user-space DSM socket stack that reduces network I/O while preserving Linux socket compatibility [13]; an RDMA+CXL hybrid disaggregation architecture improving scalability and efficiency [14]; a fault-tolerant DSM for partial failures [15]; a PCIe-NTB communication layer enabling parallel exchange with MPICH integration [16]; and rPCIBench for routable-PCIe fabrics to analyze flows and bottlenecks [17].

B. Hot Page Management in NUMA Architectures

NUMA partitions a system into nodes of cores and local memory, delivering lower-latency, higher-bandwidth local access while remote access incurs cross-node overhead [18]. Although scalable for HPC and large servers, imbalance and static placement can cause “NUMA skew,” degrading performance; OSes mitigate this via NUMA-aware scheduling, first-touch, and page migration [19] [20]. Figure 4 depicts a typical NUMA node organization with CPU–local-memory binding to improve locality.

In NUMA systems, hot-page-aware allocation and migration are key to optimizing performance under hybrid and tiered memory. A survey [21] summarizes key identification techniques such as access-bit sampling, hardware events, and hybrid TLB/PTE methods, analyzing their trade-offs in heterogeneous systems. Building on this, AMP [22] uses access heuristics and shadow simulation to switch among

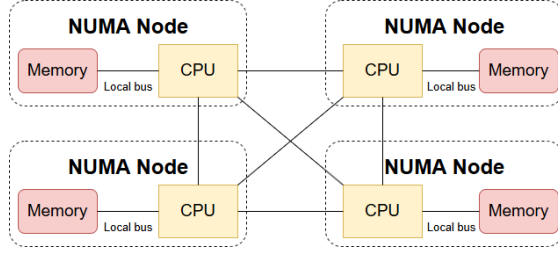


Fig. 4. NUMA Architecture Diagram

LRU, LFU, and random strategies. SWA [23] introduces sliding-window models for access trend tracking, enabling fine-grained migration control. In embedded DRAM-PCM systems, PFHA [24] uses read/write frequency ratios to preemptively migrate write-intensive regions for better endurance and energy efficiency. RHPM [25] avoids global thresholds via intra-page competition modeling, improving adaptability and reducing metadata pressure through parallel retrieval. At the system level, HeatSnap [26] accelerates VM snapshotting by prioritizing hot pages during compression in QEMU/KVM environments, while Chrono [27] leverages timer-based tracking and adaptive tuning for cold-page detection under dynamic workloads.

C. Overview of Multilayer Perceptron Applications

MLPs have recently shown strong results across domains thanks to efficient nonlinear modeling. For instance, Ding et al. [28] proposed an MLP-LightGBM hybrid model to predict hard drive failures while maintaining transferability across device models. Other researchers [29] [30] have employed MLP-based models in speech enhancement and system-level performance prediction, showing that MLPs offer competitive accuracy with faster training and inference times compared to RNNs and Transformers.

III. SYSTEM ARCHITECTURE AND RUNTIME WORKFLOW

Traditional page migration strategies often rely on fixed thresholds or heuristic rules, which lack adaptability in the face of workload variability. To address this limitation, we introduce a learning-based approach that employs a multilayer perceptron (MLP) to infer optimal migration parameters based on real-time system features. MLP is chosen for its proven effectiveness in modeling nonlinear correlations with low inference overhead. The proposed system adopts a closed-loop design comprising four modular stages—test execution, feature extraction, model training, and param-

eter prediction—linked through unified data interfaces to ensure efficient operation in CXL-enabled environments.

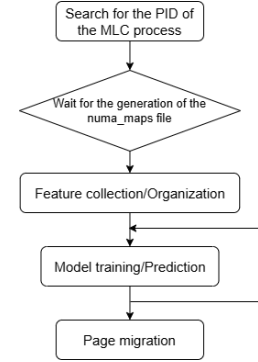


Fig. 5. The Algorithm Flow Diagram

Figure 5 shows the overall process framework of the proposed hot page migration optimization system. Figure 6 shows the logical interactions among modules. This system consists of five stages: data collection, sample accumulation, model training, parameter prediction and strategy execution. The basic contents of each part are as follows: (1) system-level runtime features are extracted using `collect_global_features_ext.py`; (2) a training dataset is constructed by accumulating feature samples through the training script; (3) the `train_param_model.py` script employs a multilayer perceptron model to learn and model optimal migration parameters; (4) online prediction is performed by `predict_params.py`, which generates adaptive threshold configurations for page migration; and (5) the `mlc_minimal_patched.sh` script coordinates page heat detection and executes the dynamic migration strategy, thereby closing the adaptive optimization loop through real-time adjustment.

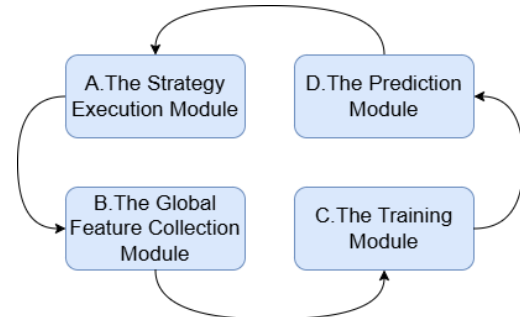


Fig. 6. The Modules Relationship Diagram

A. The Strategy Execution Module

This modular design transitions page migration from static rules to intelligent adaptation, offering a deployable and extensible solution for memory management in CXL-enabled systems. As the core of the system's control loop, the strategy execution module dynamically triggers page migration based on real-time model outputs. It is implemented via an enhanced script that integrates hot page detection, parameter prediction, and migration execution. At runtime, the script monitors `/proc/PID/numa_maps` to identify imbalanced pages, uses `predict_params.py` to generate migration thresholds, and invokes a migration utility to move pages across NUMA nodes. The process runs periodically, enabling adaptive and automated control. This design replaces static heuristics with a learning-based, modular strategy suitable for CXL-enabled memory systems.

B. The Global Feature Collection Module

The global feature acquisition module continuously instruments NUMA-level memory traces, extracting quantitative indicators for architectural-level access profiling from running MLC processes. It collects six key indicators—`total_pages`, `avg_n0`, `avg_n1`, `max_delta`, `high_delta_pages`, and `imbalance_ratio`—which reflect system load, node access concentration, access imbalance, and memory hotness distribution. These features are saved in CSV format and merged into a unified dataset for model training. For example, `total_pages` reflects load scale, `avg_nx` indicates traffic concentration, `max_delta` reveals hot-cold page differences, and `imbalance_ratio` shows node imbalance (0 means balanced, 1 means heavily skewed). After feature extraction and normalization, the system proceeds to model training, where a multi-layer perceptron (MLP) learns the mapping from feature vectors to migration parameters, enhancing its ability to generate adaptive strategies across workloads.

C. The Training Module

The training data is collected under varied workloads, and migration parameter labels are obtained by sweeping configurations for optimal performance. A 6-dimensional feature vector is extracted from the training dataset as $\mathbf{X} = [x_1, x_2, \dots, x_6]$, where x_1 denotes the total number of memory pages (`total_pages`), x_2, x_3 represent the average number of page accesses on Node 0/1 (`avg_n0`,

TABLE I
FEATURE SEMANTICS FOR PAGE MIGRATION

Feature	Meaning
Total_pages	Total number of memory pages allocated during MLC test
Avg_n0	Average access count on local DRAM (Node 0)
Avg_n1	Average access count on CXL device (Node 1)
Max_delta	Maximum access difference among all pages
High_delta_pages	Count of pages exceeding access difference threshold
Imbalance_ratio	Degree of access imbalance across NUMA nodes

`avg_n1`), x_4 is the maximum page access difference (`max_delta`), x_5 indicates the number of high-difference pages (`high_delta_pages`), and x_6 characterizes the imbalance ratio of memory distribution (`imbalance_ratio`). The target output is a 3-dimensional parameter vector $Y = [y_{\max \text{ mig}}, y_{\text{threshold}}, y_{\text{interval}}]$. Z-score normalization is applied to the input features:

$$\mathbf{x}_{\text{scaled}} = \frac{\mathbf{X} - \mu_{\mathbf{X}}}{\sigma_{\mathbf{X}}}$$

where $\mu_{\mathbf{X}}$ and $\sigma_{\mathbf{X}}$ are the mean and standard deviation of the features in the training set, respectively. For small-sample scenarios, adaptive training mode is enabled to avoid evaluation bias caused by validation set partitioning. A fully connected neural network with hierarchical representation capacity is adopted as the prediction engine within the system architecture, optimizing migration control parameters at runtime, with the following topology:

$$f(\mathbf{X}) = W_3 \cdot \sigma(W_2 \cdot \sigma(W_1 \cdot \mathbf{X} + b_1) + b_2) + b_3$$

where $W_1 \in \mathbb{R}^{6 \times 32}$, $W_2 \in \mathbb{R}^{32 \times 16}$, and $W_3 \in \mathbb{R}^{16 \times 3}$ are weight matrices, and $\sigma(\cdot)$ denotes the ReLU activation function. Weights are initialized using He normal distribution and biases are initialized to zero. The mean squared error loss function is defined as:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{Y}_i - f(\mathbf{X}_i)\|_2^2$$

The Adam optimizer is adopted for parameter updates, with an initial learning rate $\alpha = 10^{-3}$ and batch size set to the full dataset (for $N \geq 10$, an 8:2 split is used for training and validation sets). A

total of 200 training iterations are executed, and the validation loss $\mathcal{L}_{val}$ is computed every 20 epochs to monitor the model's generalization performance. Upon completion, the trained model weights are saved in the file `parampredictor.pt`, along with the corresponding StandardScaler object to ensure consistency during the inference phase.

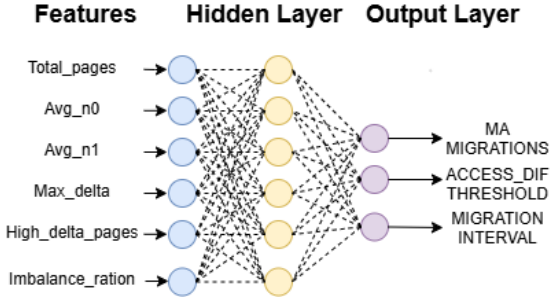


Fig. 7. Architecture of the MLP-Based Parameter Prediction Model

Figure 7 depicts the MLP-based prediction model used for dynamic page migration control. It takes six runtime features as input and outputs three key scheduling parameters to guide migration decisions in heterogeneous memory systems.

D. The Prediction Module

In the prediction stage, a 6-dimensional feature vector is standardized and input to the trained neural network, which outputs three migration parameters: `MAX_MIGRATIONS`, `ACCESS_DIFF_THRESHOLD`, and `MIGRATION_INTERVAL`. A lower-bound safeguard ensures output stability under uncertainty. The results are printed and used by the main script to enable real-time, adaptive migration policy adjustment based on memory access behavior.

IV. EXPERIMENTAL FRAMEWORK

The experiment was conducted on a host with the 11th generation Intel Core i7-11700 processor (4 cores, 2 NUMA nodes), running Ubuntu 22.04.5 LTS and Linux kernel 6.8, and a dual-node NUMA environment based on QEMU was established. Conduct page migration tests by simulating local DRAM and remote CXL memory. Figure 8 illustrates the virtual NUMA architecture used in the experiments.

A. Comprehensive Performance Comparison

To evaluate the adaptability of the proposed page migration strategy under heterogeneous memory pressure, we select three representative memory access

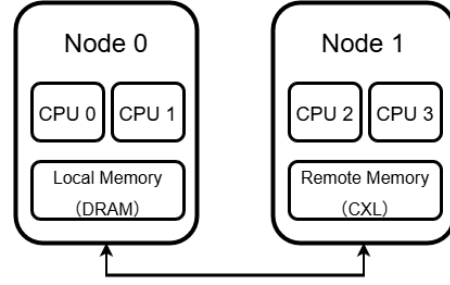


Fig. 8. The NUMA Architecture of This Experiment

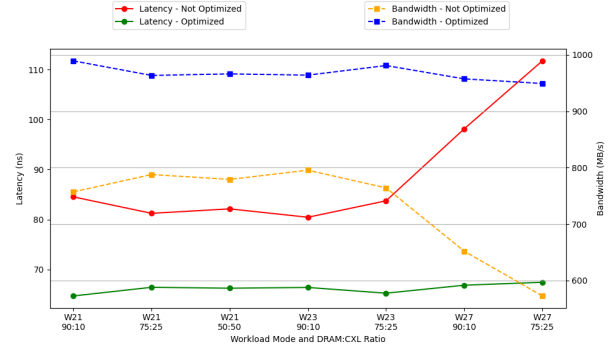


Fig. 9. Performance Comparison With and Without Migration Optimization

patterns—W21, W23, and W27—predefined in the Intel Memory Latency Checker (MLC) benchmarking tool. Specifically, W21 issues 100% sequential read operations and represents a read-intensive workload. W23 introduces a 3:1 read/write ratio, modeling a mixed access scenario. W27 applies two reads followed by a non-temporal write, emulating a bandwidth- and latency-sensitive pattern. These synthetic workloads capture typical access characteristics observed in parallel and distributed systems and serve as controlled benchmarks for performance comparison under varying DRAM-CXL ratios.

Figure 9 shows the changes in latency and bandwidth of the system with and without the optimization strategy under different workloads (W21, W23, W27) and different DRAM:CXL ratio configurations (90:10 and 75:25).

B. Comparison of Dynamic And Static Strategies

To further analyze the influence of hot page determination parameters on migration performance, this paper introduces `ACCESS_DIFF_THRESHOLD` as one of the model prediction outputs, and conducts comparative experiments under three discrete thresholds of 10, 20, and 30. Figures 10 and 11 respectively evaluate the

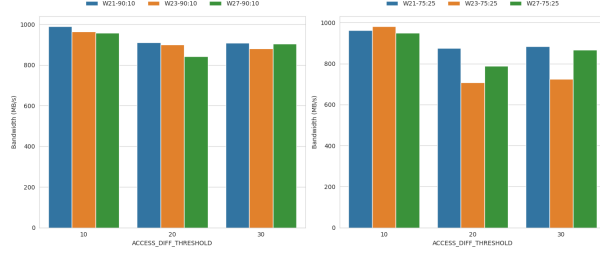


Fig. 10. Bandwidth Impact of Varying Thresholds Under Skewed Workloads

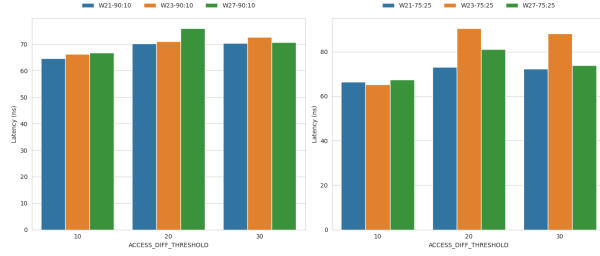


Fig. 11. Latency Impact of Varying Thresholds Under Skewed Workloads

influence of this parameter setting on three types of loads from the perspectives of latency and bandwidth. The experiment found that when the threshold was set at 10, the latency was the lowest (about 63-65ns) and the bandwidth was maximized (about 990MB/s), while when the threshold was increased to 30, performance degradation was caused in some scenarios. This verifies that the heat difference threshold predicted by the model has strong pertinence and generalization ability, and can dynamically generate a better configuration according to the specific load, avoiding the errors and limitations caused by manual setting.

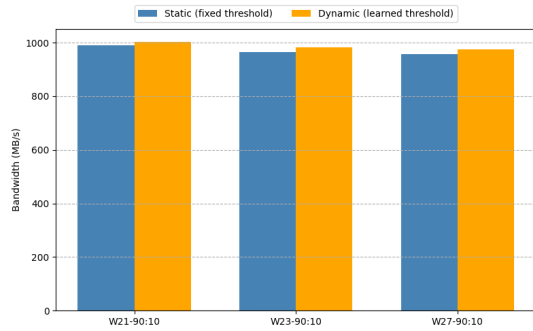


Fig. 12. Bandwidth Comparison at a ratio of 75:25

In this study, we compare and analyze the performance difference between the static threshold-based page migration policy and the dynamic migration pol-

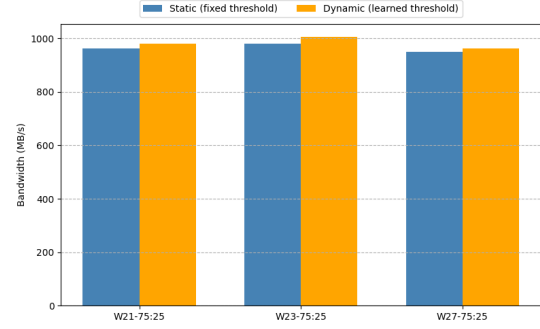


Fig. 13. Bandwidth Comparison at a ratio of 90:10

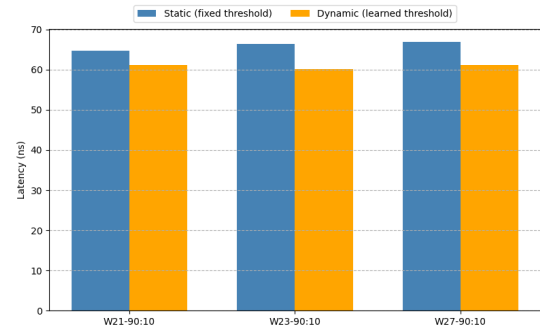


Fig. 14. Latency Comparison at a ratio of 75:25

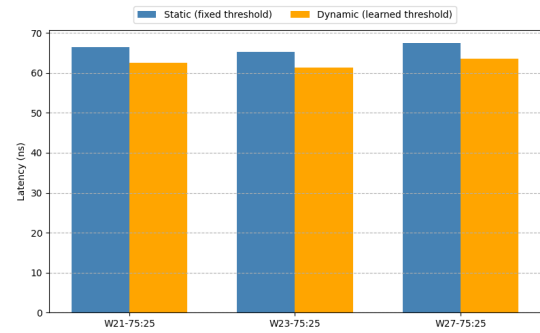


Fig. 15. Latency Comparison at a ratio of 90:10

icy with the introduction of machine learning prediction model under different workloads and DRAM:CXL configuration ratios. Three different workload modes W21, W23, and W27 are used in the experiments, and two DRAM:CXL ratios of 90:10 and 75:25 are configured to comprehensively evaluate the performance of the two strategies in the bandwidth and latency dimensions.

When the DRAM:CXL ratio is 90:10, the average latency of the dynamic strategy is 60.81ns, which is 5.17ns lower than the average latency of the static strategy 65.98ns. At the ratio of 75:25, the latency

of the dynamic strategy is 62.49 ns, which is 3.88ns lower than that of the static strategy of 66.37 ns. In terms of bandwidth, the dynamic strategy reaches 985.93 MB/s at 90:10, which is 15.80 MB/s higher than the static strategy 970.13 MB/s. At 75:25, dynamic reaches 983.03 MB/s, which is 18.56 MB/s higher than static's 964.47 MB/s. In addition, the dynamic strategy shows comprehensive advantages in all specific workloads (W21, W23, W27), especially in read-intensive workloads (W23-90:10), the latency reduction is the most significant (6.31 ns reduction, 9.5% reduction). The bandwidth improvement is most prominent in the scenario of increasing write load (W23-75:25) (24.2 MB/s, 2.5% increase), indicating that the dynamic strategy has stronger adaptability to the changes of workload characteristics.

The performance of the dynamic policy depends on the workload characteristics collected in the early stage to train the model, and the performance of the dynamic policy will fluctuate in the early stage of training. However, after more rounds of training and learning, the performance of the dynamic policy will exceed the performance of the static policy.

C. Conclusion of Experiments

Focusing on optimizing hot page migration in CXL-extended memory environments, this study systematically verifies the effectiveness of a dynamic strategy based on a multi-layer perceptron (MLP) prediction model. A dual-NUMA-node QEMU platform, combined with Intel MLC profiling, enables comprehensive evaluation under diverse workloads (W21, W23, W27) and DRAM:CXL ratios (90:10 and 75:25). Results show that while the dynamic strategy exhibits an initial performance gap, it gradually stabilizes through iterative learning and approaches optimal configurations under several workloads. Overall, the system achieves stable latency and bandwidth across scenarios, validating the proposed closed-loop learning framework.

V. CONCLUSION AND PROSPECT

This paper proposes a learning-based hot page migration framework for CXL-extended memory systems, using a multilayer perceptron (MLP) to predict migration parameters from runtime features in a closed-loop architecture. Experiments on the dual numa QEMU platform show that compared with the performance of the static method, the dynamic migration gradually improves through adaptive learning to meet the optimal

parameters in the current environment under different workloads and DRAM:CXL ratio.

While the current implementation is limited to virtual environments and training data diversity, future work will explore deployment on real CXL hardware and extend the framework with reinforcement learning and OS-level memory coordination. This system provides a scalable and intelligent approach to page scheduling in disaggregated memory architectures.

REFERENCES

- [1] K. Sasaki, M. Hanai, K. Miyazawa, A. Kobayashi, N. Oda and S. Yamaguchi, "TCP Fairness Among Modern TCP Congestion Control Algorithms Including TCP BBR," 2018 IEEE 7th International Conference on Cloud Networking (CloudNet), Tokyo, Japan, 2018, pp. 1-4, doi: 10.1109/Cloud-Net.2018.8549505.
- [2] H. N. Aung and H. Ohsaki, "On the Potential of Modern TCP Congestion Control Algorithms in Information-Centric Networking," 2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC), Torino, Italy, 2023, pp. 1107-1112, doi: 10.1109/COMPSAC57700.2023.00167.
- [3] A. Patil, V. Nagarajan, N. Nikoleris and N. Oswald, "Apta: Fault-tolerant object-granular CXL disaggregated memory for accelerating FaaS," 2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), Porto, Portugal, 2023, pp. 201-215, doi: 10.1109/DSN58367.2023.00030.
- [4] Chengsong Tan, Alastair F. Donaldson, and John Wickerson. 2025. Formalising CXL Cache Coherence. In Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '25). Association for Computing Machinery, New York, NY, USA, 437–450. <https://doi.org/10.1145/3676641.3715999>.
- [5] Kwanghoon Choi, Igjae Kim, Sunho Lee, and Jaehyuk Huh. 2025. ShieldCXL: A Practical Obliviousness Support with Sealed CXL Memory. ACM Trans. Archit. Code Optim. 22, 1, Article 13 (March 2025), 25 pages. <https://doi.org/10.1145/3703354>.
- [6] Q. Yang, R. Jin, B. Davis, D. Inupakutika and M. Zhao, "Performance Evaluation on CXL-enabled Hybrid Memory Pool," 2022 IEEE International Conference on Networking, Architecture and Storage (NAS), Philadelphia, PA, USA, 2022, pp. 1-5, doi: 10.1109/NAS55553.2022.9925356.
- [7] Jianguo Liu and Zhibin Yu. 2024. Global-State Aware Automatic NUMA Balancing. In Proceedings of the 15th Asia-Pacific Symposium on Internetwork (Internetwork '24). Association for Computing Machinery, New York, NY, USA, 317–326. <https://doi.org/10.1145/3671016.3671380>.
- [8] Wenda Tang, Tianxiang Ai, and Jie Wu. 2024. Tiresias: Optimizing NUMA Performance with CXL Memory and Locality-Aware Process Scheduling. In Proceedings of the ACM Turing Award Celebration Conference - China 2024 (ACM-TURC '24). Association for Computing Machinery, New York, NY, USA, 6–11. <https://doi.org/10.1145/3674399.3674411>.
- [9] X. Lin, J. Peng, R. Liu and X. Gao, "Research on the CXL Memory," 2024 IEEE 6th Advanced Information Management, Communicates, Electronic and Automation Control

- Conference (IMCEC), Chongqing, China, 2024, pp. 1428-1432, doi: 10.1109/IMCEC59810.2024.10575852.
- [10] R. Abdullah, H. Lee, H. Zhou and A. Awad, "Salus: Efficient Security Support for CXL-Expanded GPU Memory," 2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA), Edinburgh, United Kingdom, 2024, pp. 1-15, doi: 10.1109/HPCA57654.2024.00027.
 - [11] W. Kwon, "ETRI Extended Memory Pool System Architecture using Gen-Z Protocol," 2021 International Conference on Information and Communication Technology Convergence (ICTC), Jeju Island, Korea, Republic of, 2021, pp. 123-125, doi: 10.1109/ICTC52510.2021.9620983.
 - [12] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. 2023. Pond: CXL-Based Memory Pooling Systems for Cloud Platforms. In Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS 2023). Association for Computing Machinery, New York, NY, USA, 574–587. <https://doi.org/10.1145/3575693.3578835>.
 - [13] Wenda Tang, Ying Han, Tianxiang Ai, Guanghui Li, Bin Yu, and Xin Yang. 2024. Yggdrasil: Reducing Network I/O Tax with (CXL-Based) Distributed Shared Memory. In Proceedings of the 53rd International Conference on Parallel Processing (ICPP '24). Association for Computing Machinery, New York, NY, USA, 597–606. <https://doi.org/10.1145/3673038.3673138>.
 - [14] Zhonghua Wang, Yixing Guo, Kai Lu, Jiguang Wan, Daohui Wang, Ting Yao, and Huatao Wu. 2024. Rcmp: Reconstructing RDMA-Based Memory Disaggregation via CXL. ACM Trans. Archit. Code Optim. 21, 1, Article 15 (March 2024), 26 pages. <https://doi.org/10.1145/3634916>.
 - [15] Mingxing Zhang, Teng Ma, Jinqi Hua, Zheng Liu, Kang Chen, Ning Ding, Fan Du, Jinlei Jiang, Tao Ma, and Yongwei Wu. 2023. Partial Failure Resilient Memory Management System for (CXL-based) Distributed Shared Memory. In Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23). Association for Computing Machinery, New York, NY, USA, 658–674. <https://doi.org/10.1145/3600006.3613135>.
 - [16] S. Pickartz, P. Reble, C. Clauss and S. Lankes, "SWIFT: A Transparent and Flexible Communication Layer for PCIe-Coupled Accelerators and (Co-)Processors," *2014 IEEE International Parallel & Distributed Processing Symposium Workshops*, Phoenix, AZ, USA, 2014, pp. 371-380, doi: 10.1109/IPDPSW.2014.48.
 - [17] Hou, Wentao, Jie Zhang, Zeke Wang and Ming Liu. "Understanding Routable PCIe Performance for Composable Infrastructures." Sympo sium on Networked Systems Design and Implementation (2024).
 - [18] A. Cho and A. Daglis, "StarNUMA: Mitigating NUMA Challenges with Memory Pooling," 2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO), Austin, TX, USA, 2024, pp. 997-1012, doi: 10.1109/MICRO61859.2024.00077.
 - [19] M. Kurnosov and E. Tokmasheva, "Optimizing Barrier Algorithms on Asymmetric Subsystems of NUMA Machines," 2021 Ural Symposium on Biomedical Engineering, Radioelectronics and Information Technology (USBEREIT), Yekaterinburg, Russia, 2021, pp. 0345-0349, doi: 10.1109/USBEREIT51232.2021.9455093.
 - [20] D. He, L. Chen, J. Liang, C. Kang and J. Bai, "NUMA-Aware Contention Scheduling on Multicore Systems," 2021 16th International Conference on Intelligent Systems and Knowledge Engineering (ISKE), Chengdu, China, 2021, pp. 452-457, doi: 10.1109/ISKE54062.2021.9755368.
 - [21] L. Yue, T. Wu, Y. Shen and J. Zhang, "A Review of Memory Management Mechanisms Based on Hot Page Monitoring," 2024 3rd International Conference on Artificial Intelligence and Computer Information Technology (AICIT), Yichang, China, 2024, pp. 1-4, doi: 10.1109/AICIT62434.2024.10730021.
 - [22] T. Heo, Y. Wang, W. Cui, J. Huh and L. Zhang, "Adaptive Page Migration Policy With Huge Pages in Tiered Memory Systems," in IEEE Transactions on Computers, vol. 71, no. 1, pp. 53-68, 1 Jan. 2022, doi: 10.1109/TC.2020.3036686.
 - [23] N. Niu, X. Li, F. Fu, F. Lai, B. Yang and J. Wang, "SWA: A Novel Page Migration Scheme For Hybrid Memory System," 2021 IEEE 4th International Conference on Computer and Communication Engineering Technology (CCET), Beijing, China, 2021, pp. 203-206, doi: 10.1109/CCET52649.2021.9544355.
 - [24] N. Niu et al., "PFHA: A Novel Page Migration Algorithm for Hybrid Memory Embedded Systems," in IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 29, no. 10, pp. 1685-1692, Oct. 2021, doi: 10.1109/TVLSI.2021.3090476.
 - [25] Z. Peng, D. Feng, J. Chen, J. Hu and C. Huang, "RHPM: Using Relative Hotness to Guide Page Migration for Hybrid Memory Systems," in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 42, no. 8, pp. 2514-2526, Aug. 2023, doi: 10.1109/TCAD.2022.3231836.
 - [26] Kangyue Gao, Chuangyu Ouyang, Xinkui Zhao, Miao Ye, Chen Zhi, Guanjie Cheng, Yueshen Xu, Shuiguang Deng, and Jianwei Yin. 2025. HeatSnap: A Hot Page-Aware Continuous Snapshots System for Virtual Machines in Web Infrastructure. In Proceedings of the ACM on Web Conference 2025 (WWW '25). Association for Computing Machinery, New York, NY, USA, 3598–3606. <https://doi.org/10.1145/3696410.3714824>.
 - [27] Zhenlin Qi, Shengan Zheng, Ying Huang, Yifeng Hui, Bowen Zhang, Linpeng Huang, and Hong Mei. 2025. Chrono: Meticulous Hotness Measurement and Flexible Page Migration for Memory Tiering. In Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25). Association for Computing Machinery, New York, NY, USA, 835–853. <https://doi.org/10.1145/3689031.3717462>
 - [28] J. Ding, H. Li and J. Li, "MLP-LightGBM Hard Drive Failure Prediction with Transfer Learning Performance," 2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA), Kaifeng, China, 2024, pp. 1333-1340, doi: 10.1109/ISPA63168.2024.00179.
 - [29] C. Lyu, M. Nourazar and B. Goossens, "Predicting Inference Time and Energy Consumption in Deep Learning Using MLPs," 2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA), Kaifeng, China, 2024, pp. 990-996, doi: 10.1109/ISPA63168.2024.00131.
 - [30] H. Song, M. Kim and J. W. Shin, "Speech Enhancement Using MLP-Based Architecture With Convolutional Token Mixing Module and Squeeze-and-Excitation Network," in IEEE Access, vol. 10, pp. 119283-119289, 2022, doi: 10.1109/ACCESS.2022.3221440.